Exercício Programa 1 - Algoritmos e Estrutura de Dados II (2026)

Professora Ariane Machado Lima

Prazo de entrega: 03/05/2026 às 23:59h

1. Introdução

Os grafos são estruturas muito versáteis, com inúmeras aplicações que podemos ver no nosso dia a dia. Uma delas é a conexão entre componentes de uma rede de computadores! Conforme vamos estudando o funcionamento das redes, desde aplicações de software até detalhes sobre o hardware necessário, encontramos o plano de controle da rede: a lógica que controla a maneira como a informação viaja por vários componentes, desde a origem até o destino. Para isso, são usados vários algoritmos de roteamento, que têm como objetivo determinar caminhos bons, o que normalmente significa menor custo possível.

Empolgado em conectar as unidades produtivas de seu novo cliente, um consultor de infraestrutura foi contratado por uma empresa que possui diversas fábricas e decidiu mapear e conectar toda a rede para demonstrar à diretoria como funcionaria a comunicação entre os diferentes servidores via cabeamento de fibra óptica. Infelizmente, calcular manualmente todas as rotas possíveis entre as fábricas consome muito tempo, e o consultor ainda possui outros projetos complexos em andamento. Então, neste EP você vai ajudá-lo nessa missão, implementando um algoritmo de roteamento que conectará as unidades de produção.

2. Objetivo

Ao desenvolver este exercício-programa, você deverá implementar o algoritmo de Prim para obter uma árvore geradora mínima (AGM), **utilizando um heap binário mínimo como fila de prioridade**.

Para simular a rede de fibra óptica, o consultor mapeará os servidores das fábricas com um **arquivo de entrada de formato .txt**. Como os componentes da rede são os vértices do nosso grafo **não direcionado** e as possíveis conexões via cabeamento são as arestas com seus pesos, o grafo **será lido por meio desse arquivo**, que poderá ser facilmente modificado para incluir novas unidades produtivas conforme necessário.

Depois, **você deverá verificar se o grafo é conectado**, ou seja, se cada vértice é alcançável a partir de qualquer outro vértice, pois este é um pré-requisito para se obter uma AGM. Por fim, aplique o algoritmo de Prim para obter a árvore geradora mínima do grafo e **registre o resultado em um arquivo de saída de formato .txt.**

O programa também deverá funcionar tanto para implementação de grafos usando matriz de adjacência quanto lista de adjacência, de **forma modular como discutido em aula** (detalhes abaixo na especificação dos arquivos esperados). Qual implementação usar (matriz ou lista) **deverá ser definida no momento da compilação**. Por isso, a aplicação deverá utilizar, no código, diretivas do compilador (#ifdef), como visto em aula e definido abaixo. Dessa forma, você deverá entregar **pelo menos três e no máximo cinco arquivos (limitação máxima do próprio Edisciplinas)** com as seguintes especificações:

- 1 arquivo com a implementação de grafos por matriz de adjacência (grafo_matrizadj.c) (contendo apenas a definição da estrutura de dados e a implementação das funções de interface para matriz de adjacência);

- 1 arquivo com a implementação de grafos por lista de adjacência (grafo_listaadj.c) (contendo apenas a definição da estrutura de dados e a implementação das funções de interface para lista de adjacência);

- 1 arquivo contendo a função main (ep1_XXXX.c, em que XXXX é seu nº USP) e as funções para verificar se o grafo de entrada é isento de erros e conectado, além da construção da AGM utilizando o algoritmo de Prim, **incluindo a manipulação do heap binário mínimo** (tudo utilizando apenas as funções definidas na interface de grafos, ou seja, sem assumir explicitamente a estrutura por matriz ou lista de adjacência). Este arquivo deverá incluir, em suas primeiras linhas, os #include's dos arquivos .c das implementações de grafos exatamente desta maneira:

```
#ifdef MATRIZ
#include "grafo_matrizadj.c"
#else
#include "grafo_listaadj.c"
#endif
```

Você pode, se quiser, modularizar em mais arquivos (por ex: um arquivo adicional para implementação de heaps), desde que o EP possa ser compilado e executado conforme definido na seção 2.3, e respeitadas as observações definidas na seção 3.

2.1 Especificações de entrada

Na primeira linha, você encontrará dois valores: a quantidade V de vértices e a quantidade A de arestas.

As próximas "A" linhas correspondem às arestas do grafo. Cada linha terá três valores separados por um espaço: os dois primeiros são os números dos vértices que definem a aresta, e o terceiro é o peso da aresta. Esse peso representa o custo gerado pelo atraso de propagação, isto é, a distância física entre os aparelhos, e deverá ser tratado como um valor de ponto flutuante (tipo float); por representarem distâncias físicas, não devem ser aceitos valores negativos. Da mesma forma, como os vértices representam locais existentes para o mapeamento, não deve ser aceita a criação de arestas para vértices inexistentes, tampouco vértices negativos.

Além disso, não deve haver arestas paralelas entre os mesmos vértices, e nenhuma aresta deve resultar em auto-laço (**tudo isso deverá ser verificado em seu programa!**).

Você deverá inserir as arestas no grafo na mesma ordem em que elas forem descritas no arquivo!

Exemplo de entrada correta:

---

6 10

0 1 6.3

0 2 1.7

0 3 5.4

1 2 2.9

1 4 5.5

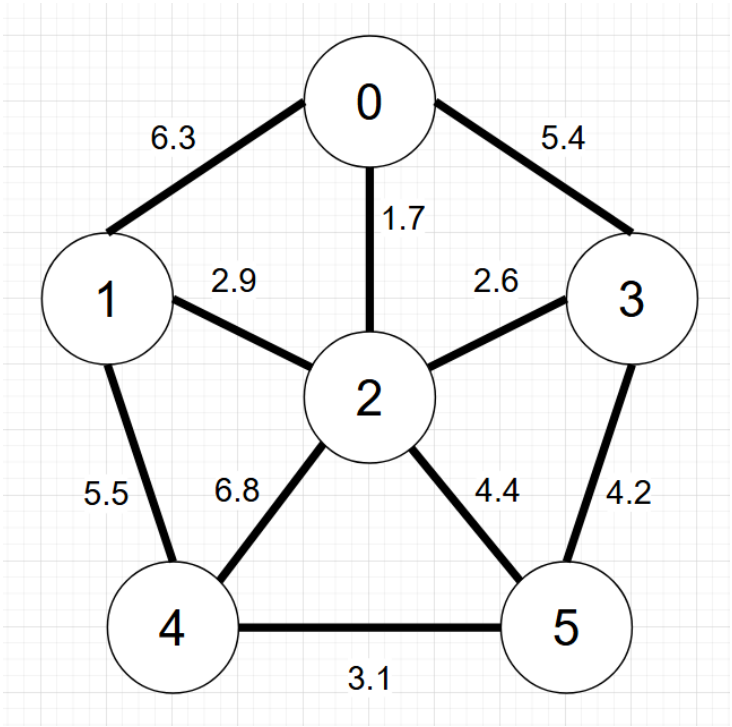
2 3 2.6

2 4 6.8

2 5 4.4

3 5 4.2

4 5 3.1



2.2 Especificações de saída

A saída do EP **deve ser impressa em um arquivo de saída de formato .txt** (ver especificações na seção 2.3). Caso o grafo possua irregularidades que impeçam o devido funcionamento do algoritmo de Prim, o arquivo de saída deverá descrever **todos os erros (dos previstos a seguir) presentes no arquivo**, ou seja, deverá conter uma ou mais das seguintes mensagens:

- "ERRO: VERTICE INVALIDO (<nr inválido do vértice>)", no caso de tentar criar uma aresta com um vértice de número inválido (o número inválido deve ser impresso entre parêntesis);
- "ERRO: AUTO-LACO (<vértice>,<vértice>)", no caso de uma aresta conectando um vértice a si próprio (o número do vértice que compõe o auto-laço (*self-loop*), separados por vírgula sem espaço, devem ser impressos entre parêntesis);
- "ERRO: ARESTA PARALELA (<vértice de menor número>,<vértice de maior número>)", no caso de duas ou mais arestas entre dois vértices (os números dos vértices que

compõem a aresta paralela, separados por vírgula sem espaço, devem ser impressos entre parêntesis, em ordem crescente);

- “ERRO: PESO INVALIDO (<peso>)”, no caso de peso negativo ou nulo (o valor do peso deve ser impresso entre parêntesis);
- “ERRO: GRAFO NAO CONECTADO”, no caso de um grafo desconexo;

Note que a mensagem de erro precisa ser **EXATAMENTE** como acima (letras maiúsculas, sem acentuação, com ”:”).

Durante a leitura do arquivo de entrada, o programa deve verificar a ocorrência de pesos inválidos, arestas paralelas, auto-laços e vértices inválidos, acumulando as devidas mensagens de erro correspondentes e **escrevendo-as no arquivo de saída, uma por linha, na ordem em que foram detectadas, ou seja, primeiro na ordem por linha do arquivo de entrada, e depois seguindo a ordem dos erros descritos acima**.

Para os erros detectados, exceto GRAFO NAO CONECTADO, a mensagem deve identificar o vértice inválido, os vértices em que ocorreram auto-laço e/ou aresta paralela e o peso inválido, de acordo com o erro encontrado.

No caso de caso uma mesma aresta possuir múltiplas irregularidades, **todos os erros devem ser reportados, seguindo a seguinte ordem de prioridade: vértice inválido, auto-laço, aresta paralela e peso inválido**.

Caso ao menos um desses erros seja encontrado, o programa deve encerrar sem executar o algoritmo de Prim, e a **verificação de conectividade do grafo só deve ser realizada caso nenhum erro de entrada tenha sido detectado**; caso o grafo esteja correto e seja apenas desconexo, apenas a mensagem "ERRO: GRAFO NAO CONECTADO" deve ser escrita no arquivo de saída.

Caso contrário, a saída **deve apresentar um relatório da rede no exato modelo a seguir:**

O relatório de rede **deve** conter:

- Na primeira linha, o **custo total da árvore geradora mínima**, ou seja, a soma dos custos das arestas da árvore, **sempre** com uma casa decimal;
- Na segunda linha, o **vértice mais distante da origem**, isto é, o vértice que acumula a maior soma de pesos ao longo da árvore geradora mínima a partir do vértice 0, que deve ser utilizado como vértice de origem para fins de padronização da saída. Em caso de empate o menor vértice deve ser impresso;
- Da terceira linha em diante, cada linha deve apresentar cada uma das arestas com três valores separados por um espaço: primeiro o vértice de menor índice, que chamaremos aqui de vértice origem, e o outro vértice, de maior índice, que chamaremos de vértice destino, e o custo, sempre com **uma casa decimal**. As arestas devem ser impressas por ordem crescente de vértice origem e, em caso de empate, em ordem crescente de vértice destino.

Exemplo de saída referente:

14.5

4

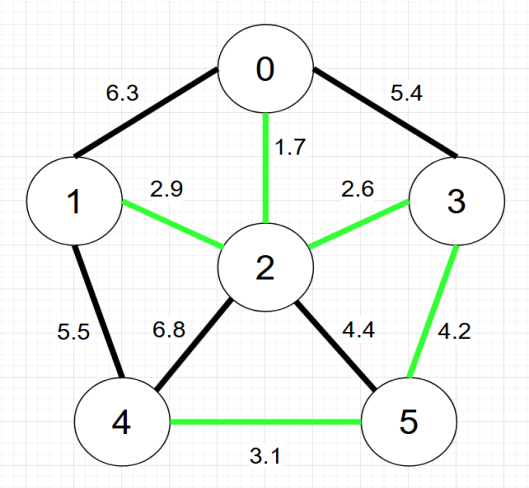
0 2 1.7

1 2 2.9

2 3 2.6

3 5 4.2

4 5 3.1



2.3 Compilação e execução

Seu programa deve funcionar tanto para a implementação usando matriz de adjacência quanto para a implementação usando lista de adjacência. Para facilitar o processo de teste, informaremos qual implementação será utilizada (matriz ou lista) via diretiva do compilador. Para isso, **você deverá entregar entre 3 e 5 arquivos**, conforme especificado na seção 2 (Objetivo).

- O programa deve ser compilado via linha de comando utilizando o gcc, substituindo XXXX pelo seu número USP em todos os comandos.
- Para compilar com lista de adjacência: **gcc -o ep1_XXXX ep1_XXXX.c -lm**
- Para compilar com matriz de adjacência: **gcc -DMATRIZ -o ep1_XXXX ep1_XXXX.c -lm**

O programa deve ser executado com o seguinte comando:

./ep1_XXXX arquivo_entrada.txt arquivo_saida.txt

Onde **arquivo_entrada é o arquivo com o grafo a ser lido** e **arquivo_saida é o arquivo onde o relatório será escrito**. Note que "**arquivo_entrada.txt**" e "**arquivo_saida.txt**" são apenas exemplos. Quaisquer nomes de arquivos podem ser passados na linha de comando. Toda a saída do programa, incluindo as mensagens de erro, deve ser escrita exclusivamente no arquivo de saída. Caso seja necessário utilizar algum parâmetro adicional de compilação, **ele deve ser informado em um arquivo README.txt** entregue junto com os demais arquivos no Edisciplinas.

3. Observações

- O código deve ser implementado na linguagem C;
- Assuma que o **número máximo** de vértices é 100;
- Note que o nome do arquivo ep1_XXXX.c deve ter no lugar do XXXX seu nº USP;
- Os arquivos (**no mínimo 3 e no máximo 5**) devem ser postados individualmente e exclusivamente no Edisciplinas (não compactá-los!), sem pastas ou subpastas, até a data final marcada;
- É **expressamente proibido ter duas versões** (uma para matriz de adjacência e outra para lista de adjacência) das funções para avaliar se um grafo é conectado, construção da AGM, etc;
- Note que as saídas para matriz de adjacência e lista de adjacência não são necessariamente iguais. Nos testes do EP haverá como gabaritos uma saída para matriz e uma saída para lista. Por isso é muito importante que você **insira as arestas na ordem em que forem descritas na entrada**;
- Os EPs que não seguirem o padrão de nomenclatura, formatação de saída e nome dos arquivos ou de qualquer outra definição desta especificação **terão sua pontuação reduzida;**
- Qualquer evidência de plágio entre trabalhos, mesmo que parcial, implicará na **nota zero** no trabalho **para todos os envolvidos!**
- A função **le_grafo deve retornar um valor booleano** indicando se a leitura do grafo foi realizada com sucesso ou não;
- Deverá ser implementada uma função de assinatura **verifica_validade_vertice**, cujo propósito é assegurar que vértices com valores negativos não serão aceitos, e ela **deve retornar um valor booleano** indicando se o vértice é válido ou não;
- Casos de teste serão fornecidos entretanto é **essencial** que o programa seja testado individualmente;

4. Avaliação

A nota atribuída ao EP será baseada nas funcionalidades solicitadas, entretanto se atente a aspectos como:

- Documentação: comentários explicando o que seu código faz;
- Identação: legibilidade do código e uso correto das chaves;
- Funcionalidade: O quanto seu código executa corretamente a tarefa solicitada;

Além disso, **erros de sintaxe e detecção de código gerados por I.A** serão tratados como impedimento de compilação e receberão nota **ZERO**.